

SYSTEM ANALYSIS DOCUMENTATION (SAD)

ScamBreaker

UMHackathon 2026

April 2026

Document Control

Field	Detail
System Under Test (SUT)	ScamBreaker (AI-Powered Scam Case Management Platform)
Team Repo URL	https://github.com/srthanakesh/scambreaker.git
AI Model Used	ZhipuAI GLM (Z AI Model) - GLM-5.1 / GLM-4.6v / GLM-4.5-air
Document Version	v1.0 - Preliminary Round Submission

1. Introduction

1.1 Purpose

This System Analysis Document (SAD) describes the technical architecture, data flows, system design decisions, and operational model of ScamBreaker, an AI-powered scam reporting and case management platform built for UMHackathon 2026. It serves as the authoritative technical reference for developers, testers, and judges evaluating the system's engineering depth and feasibility.

1.2 Background

ScamBreaker was conceived in response to Malaysia's accelerating financial scam crisis. Existing reporting mechanisms are fragmented across agencies, require victims to re-explain their situation multiple times, and routinely miss the critical 1-2 hour recovery window for bank freeze requests. ScamBreaker consolidates the intake, classification, documentation, and routing pipeline into a single AI-native platform, reducing victim burden and maximising authority response speed.

1.3 Target Stakeholders

Stakeholder	Role	Expectations
Scam Victim	Files reports via the conversational chat interface; uploads evidence; receives case updates and generated documents	Fast, empathetic, multi-lingual intake; immediate recoverability feedback; auto-generated formal documents
Authority Officer	Reviews and manages incoming cases via the Command Centre dashboard; transitions workflow; adds notes; routes to agencies	Prioritised case queue by urgency; real-time SLA countdowns; structured case data; audit trail
Development Team	Builds and maintains the Next.js platform, GLM integrations, and database	Clear API contracts, modular codebase, documented architecture
QA Team	Validates all user role flows, AI outputs, and system resilience	Defined scope, request/response formats, edge case coverage

2. System Architecture & Design

2.1 High-Level Architecture Overview

Type	Details
System	Next.js 16 Web Application (Server-Side Rendering + API Routes)
Architecture	Monolithic with service-layer separation; single deployment unit
Deployment	Node.js server; PostgreSQL via Prisma ORM; deployable to Vercel / self-hosted
AI Integration	ZhipuAI GLM API via HTTP calls (not SDK); service layer abstracted in glm.ts
Authentication	JWT (HS256 via jose) stored in HTTP-only cookies; bcryptjs password hashing

ScamBreaker is structured as a client-server Next.js application with three distinct logical tiers: (1) Frontend layer: React 19 components with Tailwind CSS 4 and Framer Motion, handling chat UI, evidence upload with client-side OCR, and case/authority dashboards. (2) API layer: Next.js App Router API routes handling authentication, case creation (7-step pipeline), case retrieval, and case updates. (3) Service layer: Encapsulated GLM AI service (glm.ts), recoverability engine, case automation, agency routing, and workflow state machine; all called from API routes, never directly from the frontend.

2.2 GLM as Service Layer

2.2.1 Dependency Diagram

The following describes the GLM integration architecture within ScamBreaker:

- **Prompt Construction:** System prompts are stored in `/lib/prompts/`. The ZAI system prompt instructs GLM to detect language, classify scam type, extract structured JSON, and ask one follow-up question at a time. Case analysis prompts include the victim's raw description + OCR-extracted evidence as context.
- **Context Window:** Each API call to GLM includes: (a) the system prompt (~800 tokens), (b) full conversation history (up to 8 turns), and (c) OCR-extracted evidence text (max 10,000 chars). The total context window usage per call is bounded at ~15,000 tokens, well within GLM-5.1's 128K token limit.
- **Response Parsing:** All GLM responses are processed through the `extractJsonBlock()` utility which extracts embedded JSON from the AI's text response. Type-safe Zod schemas validate the extracted JSON before it is used downstream.
- **Token Limiting:** OCR text is truncated at 2,000 characters per file (max 5 files). Long conversation histories are truncated to the last 8 messages to prevent context overflow.
- **Major API calls:** Frontend → POST `/api/chat` → `chatWithGLM()` → ZhipuAI GLM API; Frontend → POST `/api/cases` → [`restructureStatement()` + `analyzeCase()` + `generateFormalDocuments()`] → ZhipuAI GLM API; All GLM calls use `fetchWithModelFallback()` with 3-tier model chain.

2.2.2 Model Fallback Chain

Priority	Model	Use Case	Temperature
1 (Primary)	GLM-5.1	Case analysis, document generation, chat	0.1–0.7
2 (Secondary)	GLM-4.6v	Vision-capable fallback, evidence analysis	0.1–0.7
3 (Tertiary)	GLM-4.5-air	Lightweight fallback for all tasks	0.1–0.7
4 (Emergency)	mockAnalyzeCase()	Static safe fallback if all models fail	N/A

2.3 Sequence Diagram - Victim Report Submission (Core Flow)

The following describes the complete sequence for the primary user interaction:

- Step 1: Client-Side Intake: Victim opens chat UI, selects language (EN/BM/ZH/TA), types scam description. Evidence files uploaded → Tesseract.js extracts text client-side (no server upload of raw images).
- Step 2: AI Chat Loop: POST /api/chat → chatWithGLM(GLM-5.1) with system prompt + conversation history → GLM returns reply + optional JSON block. If scamType detected in JSON → isReady=true, ReadyStateCard shown. Loop repeats until sufficient data or 3+ questions asked.
- Step 3: Case Submission: Victim clicks Submit → POST /api/cases → 7-step pipeline executes:
 - (1) restructureStatement() — formal victim statement via GLM.
 - (2) analyzeCase() — full classification if no pre-analysed JSON.
 - (3) calculateRecoverability() — 0–100 score with 6 factors.
 - (4) prisma.case.create() + AuthorityTicket creation.
 - (5) logWorkflowEvent("CASE_CREATED").
 - (6) generateFollowUpTasksForCase() — upsert deduped tasks.
 - (7) generateDraftsForCase() — 5 document drafts via GLM.
- Step 4: Success: Victim redirected to /victim/report/success/[id] showing case summary, recoverability level, and access to generated documents.
- Step 5: Authority Processing: Authority dashboard (polling every 30s) shows new case sorted by recoverability. Officer opens case, reviews, transitions workflow status, case audit-logged at each step.

2.4 Technology Stack

Layer	Technology	Justification
Frontend	React 19, Tailwind CSS 4, Framer Motion	Server-side rendering for SEO; React 19 concurrent features for responsive chat UI; Tailwind for rapid, consistent styling

Layer	Technology	Justification
Backend	Next.js 16 App Router, Server Actions, API Routes	Unified full-stack framework; Server Actions for auth; API Routes for stateless case management
Database	PostgreSQL via Prisma ORM	Relational model suits case + workflow relationships; Prisma provides type-safe queries and migration management
AI Engine	ZhipuAI GLM API (Z AI Model)	Mandatory per UMHackathon 2026; GLM-5.1 provides strong multilingual reasoning; vision capability in GLM-4.6v for evidence analysis
OCR	Tesseract.js 7 + pdf.js 5	Client-side execution preserves victim document privacy; supports EN+BM+ZH+TA language packs
Authentication	JWT (jose, HS256) + bcryptjs	Stateless, lightweight auth suitable for serverless deployment; HTTP-only cookies prevent XSS token theft

3. Key Data Flows

3.1 Data Flow Diagram (DFD)

The primary data flows through ScamBreaker are as follows:

- Victim → Chat Interface → POST /api/chat → GLM API → Structured JSON analysis → Frontend ReadyStateCard
- Victim → Submit Report → POST /api/cases → [GLM: restructure + analyze + generate docs] → PostgreSQL (Case + AuthorityTicket + Evidence + FollowUpTasks + Documents) → Victim Success Page
- Authority → GET /api/cases → PostgreSQL → Dashboard (sorted by recoverability, SLA countdown) → Case Detail View
- Authority → PATCH /api/cases/[id] → Workflow validation → PostgreSQL update + WorkflowLog → Case updated
- Recoverability Engine → reads Case fields (amountLost, scamType, rawDescription, createdAt, Evidence) → computes score → stored in Case.recoverabilityScore → decays dynamically at dashboard render time

3.2 Normalised Database Schema (3NF)

The database is designed to 3NF to ensure data integrity and eliminate redundancy. Key entities and relationships:

Entity	Key Fields	Relationships
User	id (PK), email (UK), password (bcrypt), role (VICTIM AUTHORITY), fullName, icNumber, phoneNumber	1 User → many Cases (victims); no authority-case ownership (shared queue)
Case	id (PK), userId (FK), rawDescription, scamType, amountLost, urgency, priority, summary, assignedAgency, workflowStatus, recoverabilityScore, recoverabilityLevel, timeToActMinutes, documents (JSON), priorityReason (JSON)	1 Case → 1 AuthorityTicket, many Messages, many Evidence, many FollowUpTasks, many WorkflowLogs
AuthorityTicket	id (PK), caseId (FK, UK), priority, assignedAgency, status, notes	1:1 with Case; auto-created on case submission
Evidence	id (PK), caseId (FK), fileName, mimeType, dataUrl, extractedText	Many Evidence → 1 Case
FollowUpTask	id (PK), caseId (FK), title, description, status, dedupeKey (UK per case)	Many FollowUpTasks → 1 Case; dedupeKey prevents duplicate tasks across workflow transitions
WorkflowLog	id (PK), caseId (FK), event, actor, oldStatus, newStatus, metadata (JSON), createdAt	Many WorkflowLogs → 1 Case; full audit trail

4. Functional Requirements & Scope

4.1 Minimum Viable Product (MVP) Features

#	Feature	Description
1	Conversational Intake (Victim)	Chat-based report filing in EN/BM/ZH/TA with OCR evidence upload. AI asks clarifying questions and signals readiness when sufficient data is gathered.
2	AI Case Analysis (GLM)	Automated classification of scam type, urgency, priority, assigned agency, and missing information fields. Produces structured JSON for downstream processing.
3	Recoverability Engine	0–100 scoring across 6 factors with dynamic time decay. Determines SLA urgency level (CRITICAL/HIGH/MEDIUM/LOW) and estimated time-to-act window.
4	Document Generation (GLM)	Auto-generation of 5 document drafts per case: incident brief, authority summary, request-more-info template, bank dispute letter (EN), and police report (BM).

#	Feature	Description
5	Authority Command Centre	Prioritised case queue with SLA countdown timers, duplicate detection, workflow state machine, internal notes, and full audit logging.

4.2 Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Reliability	Case submission must succeed even if 2 of 3 GLM models are unavailable	3-tier model fallback chain + mockAnalyzeCase() emergency fallback; no single point of AI failure
Performance	GLM API calls must return within 15 seconds; case creation pipeline within 40 seconds	AbortController 15s timeout per GLM call; pipeline runs sequentially (not parallel) to avoid rate limiting
Token Latency	Chat responses must arrive within 5 seconds under normal load	GLM-4.5-air fallback activates at 15s timeout; frontend shows streaming loading state during GLM calls
Security	API keys must never be exposed to the client; sessions must be tamper-proof	All GLM calls execute server-side only; JWT stored in HTTP-only cookie; API routes validate session before execution
Privacy	Victim document data must not be transmitted to third-party OCR services	Tesseract.js runs 100% client-side in the browser; only extracted text (not images) is sent to the server
Cost Efficiency	Average GLM cost per session must remain below RM 0.15	Chat capped at 8 turns; OCR text truncated at 2K chars/file; document generation runs once per case; temperature=0.1 for deterministic tasks minimises retry waste
Maintainability	System must be extensible to add new agencies and scam types without code changes	Agency routing defined in agencies.ts configuration file; scam types are enum-driven with keyword arrays

4.3 Out of Scope / Future Enhancements

- Real-time WebSocket push notifications to victims on case status changes
- Native mobile application (iOS/Android); current MVP is web-only
- Live bank API integration for automated account freeze requests
- National scam intelligence dashboard aggregating patterns across cases
- MyKad digital identity verification for victim authentication
- Multi-authority collaborative case management (shared editing)

5. Monitoring, Evaluation, Assumptions & Dependencies

5.1 Technical Evaluation Strategy

5.1.1 Deployment Strategy

ScamBreaker will be deployed with a Grayscale Rollout approach. New features are first released to 10% of authority users (internal testers) with monitoring for error rates and GLM API latency. If no anomalies are detected within 24 hours, rollout proceeds to 50%, then 100%. A/B testing will be used to evaluate improvements to the recoverability scoring algorithm; comparing recovery outcomes between score-v1 and score-v2 cases.

5.1.2 Emergency Rollback & Golden Release

A Golden Release (last stable deployment) is tagged in the CI/CD pipeline. If GLM API failure rate exceeds 10% or case creation error rate exceeds 5%, an automated alert triggers. The team can execute a one-command rollback to the Golden Release via the deployment platform. All database migrations are reversible with Prisma migrate rollback.

5.1.3 Priority Matrix

Priority Level	Trigger Condition	Response Action
P0 - Critical	GLM API all 3 models returning errors; case creation failure rate >5%	Immediate emergency rollback to Golden Release; activate mockAnalyzeCase() fallback mode; notify team
P1 - High	GLM response latency >15s on >20% of calls; recoverability score calculation errors	Rollback GLM model selection to last known-good config; investigate ZhipuAI API status
P2 - Medium	Authentication failures >1%; database connection pool exhaustion	Restart affected service; increase connection pool limit; alert monitoring dashboard
P3 - Low	OCR accuracy degradation; non-critical UI rendering issues	Log for next sprint; no emergency action required

5.2 Assumptions

- Victims have access to a modern web browser (Chrome/Firefox/Safari/Edge) with JavaScript enabled.
- ZhipuAI GLM API maintains minimum 99% availability during the hackathon demonstration period.
- PostgreSQL database is accessible and has sufficient storage for case records and base64-encoded evidence files.
- Authority officers have stable internet connectivity for dashboard polling every 30 seconds.
- OCR accuracy is sufficient for printed/digital screenshots; handwritten evidence is flagged as low-confidence.

5.3 External Dependencies

Tool / Service	Purpose	Risk	Mitigation
ZhipuAI GLM API (Z AI Model)	Core AI engine for case analysis, chat, and document generation	HIGH - all AI features depend on this API	3-tier model fallback + mockAnalyzeCase() emergency mode
PostgreSQL (Prisma ORM)	Persistent storage for cases, users, evidence, audit logs	MEDIUM - database availability	Local dev DB; production on managed PostgreSQL (e.g. Supabase/Neon)
Tesseract.js + pdf.js	Client-side OCR for evidence extraction	LOW - runs in browser, no external dependency	Language pack preloading; graceful degradation if OCR fails (evidence still attachable)
Next.js / Vercel	Web framework and deployment platform	LOW - mature ecosystem	Deployable to any Node.js host; not locked to Vercel

6. Project Management & Team Contributions

6.1 Project Timeline (10-Day Sprint)

Days	Phase	Deliverables
Day 1–2	Architecture & Planning	System design, database schema, API contract definition, GLM prompt engineering strategy
Day 3–4	Core Backend	Authentication, Case API (7-step pipeline), Recoverability Engine, GLM service layer with fallback
Day 5–6	AI Integration & Frontend	Chat UI, OCR integration, ReadyStateCard, Victim Dashboard, document viewer
Day 7–8	Authority Features	Command Centre dashboard, workflow state machine, SLA countdown, duplicate detection, audit logging
Day 9	QA & Testing	Unit tests, integration tests, AI output boundary testing, CI/CD pipeline setup
Day 10	Documentation & Pitch	PRD, SAD, QA documentation, pitch deck, demo video preparation

6.2 Recommendations

- **Redis Cache:** Cache GLM responses for identical scam descriptions (e.g., same scam type + bank + amount) to reduce inference costs for duplicate reports from the same scam wave.
- **WebSocket Implementation:** Replace 30-second polling on the authority dashboard with WebSocket push for true real-time case updates; critical for CRITICAL urgency cases.
- **Vector Similarity Search:** Add pgvector extension to PostgreSQL to enable semantic similarity search across case descriptions, improving duplicate detection beyond the current exact-field matching.
- **Rate Limiting:** Add per-IP rate limiting on the /api/cases POST endpoint to prevent automated case flooding.